LAPORAN JOBSHEET
“16 - Implementasi Login Database & Multi-Role”
MATA KULIAH PEMROGRAMAN BERBASIS FRAMEWORK

M. FATIH AL GHIFARY

2341720194 / TI-3A

PROGRAM STUDI D-IV TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026

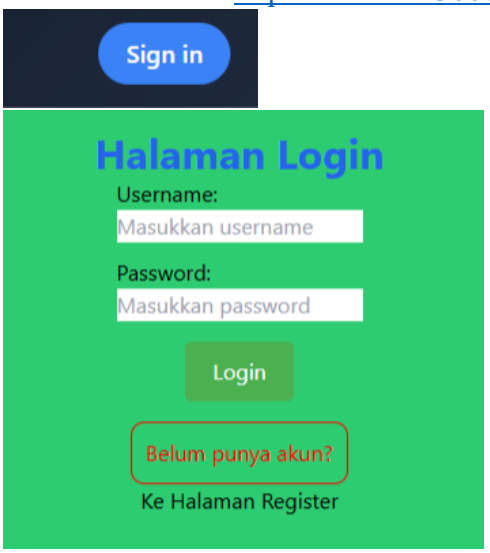
LANGKAH PRAKTIKUM

BAGIAN 1 – Custom Login Page

1. Tambahkan custom page di NextAuth line 55-57

```
P2 > Praktikum > next-routing > src > pages > api > auth > [...nextauth].ts > ...
4   export const authOptions: NextAuthOptions = {
53
54      pages: {
55          signIn: "/auth/login",
56      },
57  };
```

2. Jalankan browser http://localhost:3000/ dan klik sign in maka akan diarahkan ke login



BAGIAN 2 – Handle Login di Frontend

1. Copy paste isi dari register/index.tsx ke file login/index.tsx

```
P2 > Praktikum > next-routing > src > views > auth > login > index.tsx > ...
1   "use client";
2
3   import Link from "next/link";
4   import { useState } from "react";
5   import styles from "./register.module.scss";
6   import { useRouter } from "next/router";
7
8   const RegisterPage = () => {
9     const [isLoading, setIsLoading] = useState(false);
10    const { push } = useRouter();
11    const [error, setError] = useState("");
12
13    const [formData, setFormData] = useState({
14      fullName: "",
15      email: "",
16      password: "",
17    });
18
19    const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
20      const { name, value } = e.target;
21      setFormData((prev) => ({
22        ...prev,
23        [name]: value,
24      }));
25      // Clear error when user starts typing
```

2. Copy paste isi dari register/register.module.scss ke file login/login.module.scss

P2 > Praktikum > next-routing > src > views > auth > login > login.module.scss > ...

```scss
.register {
  min-height: 100vh;
  background: linear-gradient(135deg, #f5f7fa 0%, #c3cfe2 100%);
  padding: 2rem 1rem;
  display: flex;
  align-items: center;
  justify-content: center;

  @media (min-width: 640px) {
    padding: 2rem 1.5rem;
  }

  @media (min-width: 1024px) {
    padding: 2rem 2rem;
  }
}

.register_container {
  background-color: white;
  border-radius: 1rem;
  box-shadow: 0 20px 60px rgba(0, 0, 0, 0.08);
  overflow: hidden;
  max-width: 26rem;
  width: 100%;
  padding: 2.5rem;
```

3. Semua text register pada file index.tsx pada folder login diubah menjadi login

P2 > Praktikum > next-routing > src > views > auth > login > index.tsx > ...

```tsx
"use client";

import Link from "next/link";
import { useState } from "react";
import styles from "./login.module.scss";
import { useRouter } from "next/router";

const LoginPage = () => {
  const [isLoading, setIsLoading] = useState(false);
  const { push } = useRouter();
  const [error, setError] = useState("");

  const [formData, setFormData] = useState({
    fullName: "",
    email: "",
    password: "",
  });

  const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const { name, value } = e.target;
    setFormData((prev) => ({
      ...prev,
      [name]: value,
    }));
```

4. Jangan lupa setting link hrefnya

```tsx
<p className={styles.login_link}>
  {"Don't have an account?"}{" "}
  <Link href="/auth/register">Register here</Link>
</p>
</div>
```

5. Lakukan hal yang sama pada file login.module.scss rubah text register menjadi login

P2 > Praktikum > next-routing > src > views > auth > login > login.module.scss > .login > { } @media (min-wi

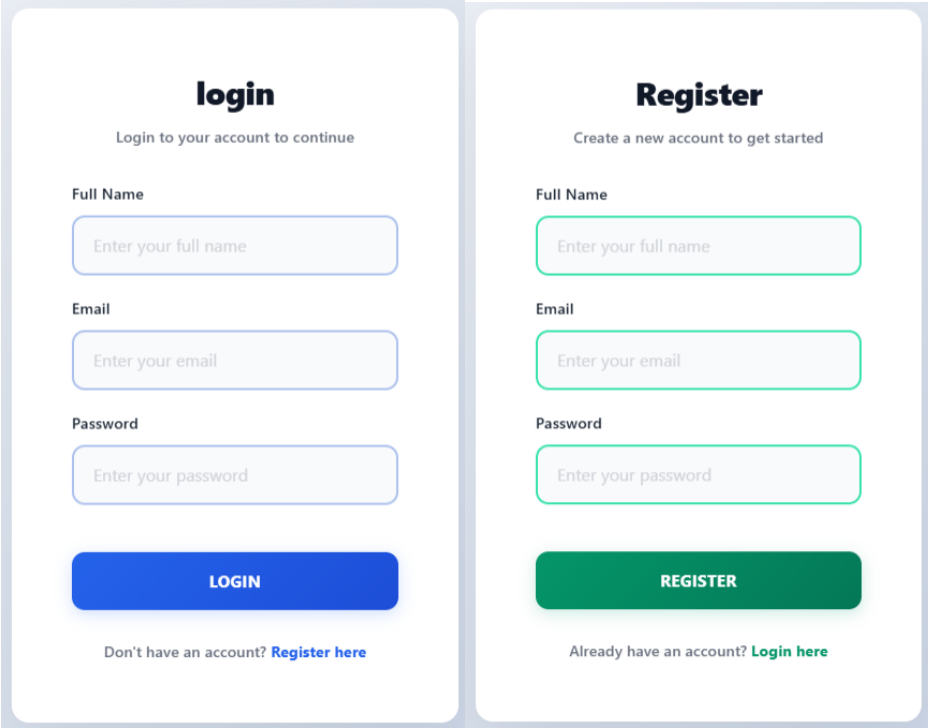
```scss
.login {
  min-height: 100vh;
  background: linear-gradient(135deg, #f5f7fa 0%, #c3cfe2 100%);
  padding: 2rem 1rem;
  display: flex;
  align-items: center;
  justify-content: center;

  @media (min-width: 640px) {
    padding: 2rem 1.5rem;
  }

  @media (min-width: 1024px) {
    padding: 2rem 2rem;
  }
}

.login_container {
  background-color: white;
  border-radius: 1rem;
  box-shadow: 0 20px 60px rgba(0, 0, 0, 0.08);
  overflow: hidden;
  max-width: 26rem;
  width: 100%;
  padding: 2.5rem;
  -webkit-backdrop-filter: blur(10px);
```

6. Cek pada file login.tsx pada pages/auth

P2 > Praktikum > next-routing > src > pages > auth > login.tsx > ...

```tsx
import TampilanLogin from "../../views/auth/login";

const HalamanLogin = () => {
  return <TampilanLogin />;
};

export default HalamanLogin;
```

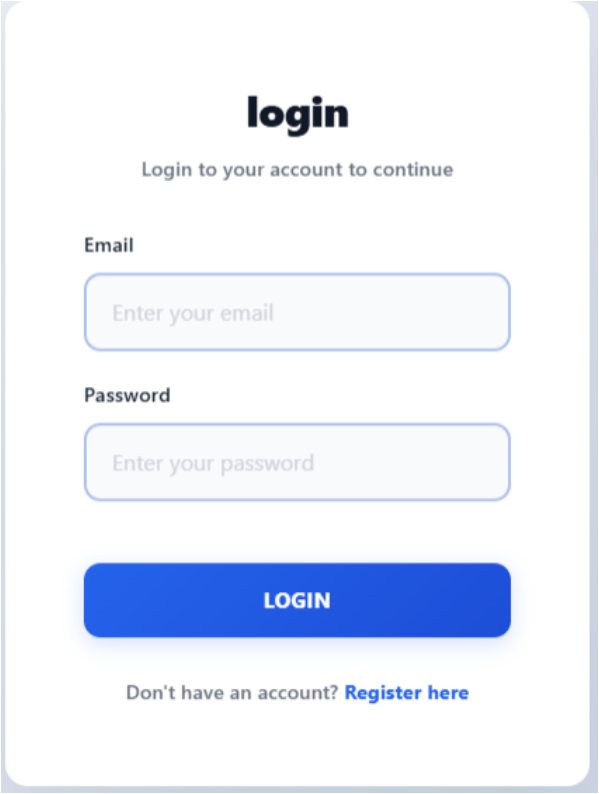
7. Jalankan browser localhost:3000/auth/login. Tampilannya akan sama dengan register



8. Pada tampilan login kita tidak perlu hapus fullname jadi pada folder views/auth/login/index.tsx hapus fullname

```
<form className={styles.login_form} onSubmit={handleSubmit}>
    <div className={styles.form_group}>
        <label htmlFor="fullName">Full Name</label>
        <input
            type="text"
            id="fullName"
            name="fullName"
            value={formData.fullName}
            onChange={handleChange}
            placeholder="Enter your full name"
            required
        />
    </div>
```

9. Sehingga hasilnya seperti berikut :



10. Buka file index.tsx pada folder views/auth/login dan modifikasi codenya seperti berikut

```
const handleSubmit = async (e: React.FormEvent<HTMLFormElement>) => {
  e.preventDefault();
  setIsLoading(true);
  setError("");
  const form = e.currentTarget;
  const formData = new FormData(form);
  const email = formData.get("email") as string;
  const password = formData.get("password") as string;

  // Validasi password minimal 6 karakter
  if (!password || password.length < 6) {
    setIsLoading(false);
    setError("Password must be at least 6 characters");
    return;
  }

  try {
    const response = await fetch("/api/login", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({ email, password }),
    });

    if (response.status === 200) {
      form.reset();
      setFormData({ email: "", password: "" });
      setIsLoading(false);
      push("/product");
    } else {
      setIsLoading(false);
      setError(
        response.status === 401
          ? "Invalid email or password"
          : "Failed to login. Please try again.",
      );
    }
  } catch (err) {
    setIsLoading(false);
    setError("An error occurred. Please try again.");
    console.error("Login error:", err);
  }
};
```

Note pastikan tulisan password pada event.password.value pada line 48 sama dengan yang ada di

```
<div className={styles.form_group}>
  <label htmlFor="password">Password</label>
  <input
    type="password"
    id="password"
    name="password"
    value={formData.password}
    onChange={handleChange}
    placeholder="Enter your password"
    required
  />
</div>
```

11. Buka file servicefirebase.ts dan tambahkan code signIn

```
export async function signIn(
  credentials: {
    email: string;
    password: string;
  },
  callback: (response: {
    status: "success" | "error";
    message: string;
    userId?: string;
    fullName?: string;
    error?: unknown;
  }) => void,
) {
  try {
    // Find user with this email
    const q = query(
      collection(db, "users"),
      where("email", "==", credentials.email),
    );
    const querySnapshot = await getDocs(q);
    const users = querySnapshot.docs.map((doc) => ({
      id: doc.id,
      ...doc.data(),
    }));

    // If user doesn't exist, return error
    if (users.length === 0) {
      callback({
        status: "error",
        message: "Invalid email or password",
      });
      return;
    }
    /* eslint-disable @typescript-eslint/no-explicit-any */
    const user = users[0] as any;
```

BAGIAN 3 – Authorize di NextAuth (Database Login)

1. Buka file [...nextauth].ts modifikasi menjadi berikut (pada bagian providers)

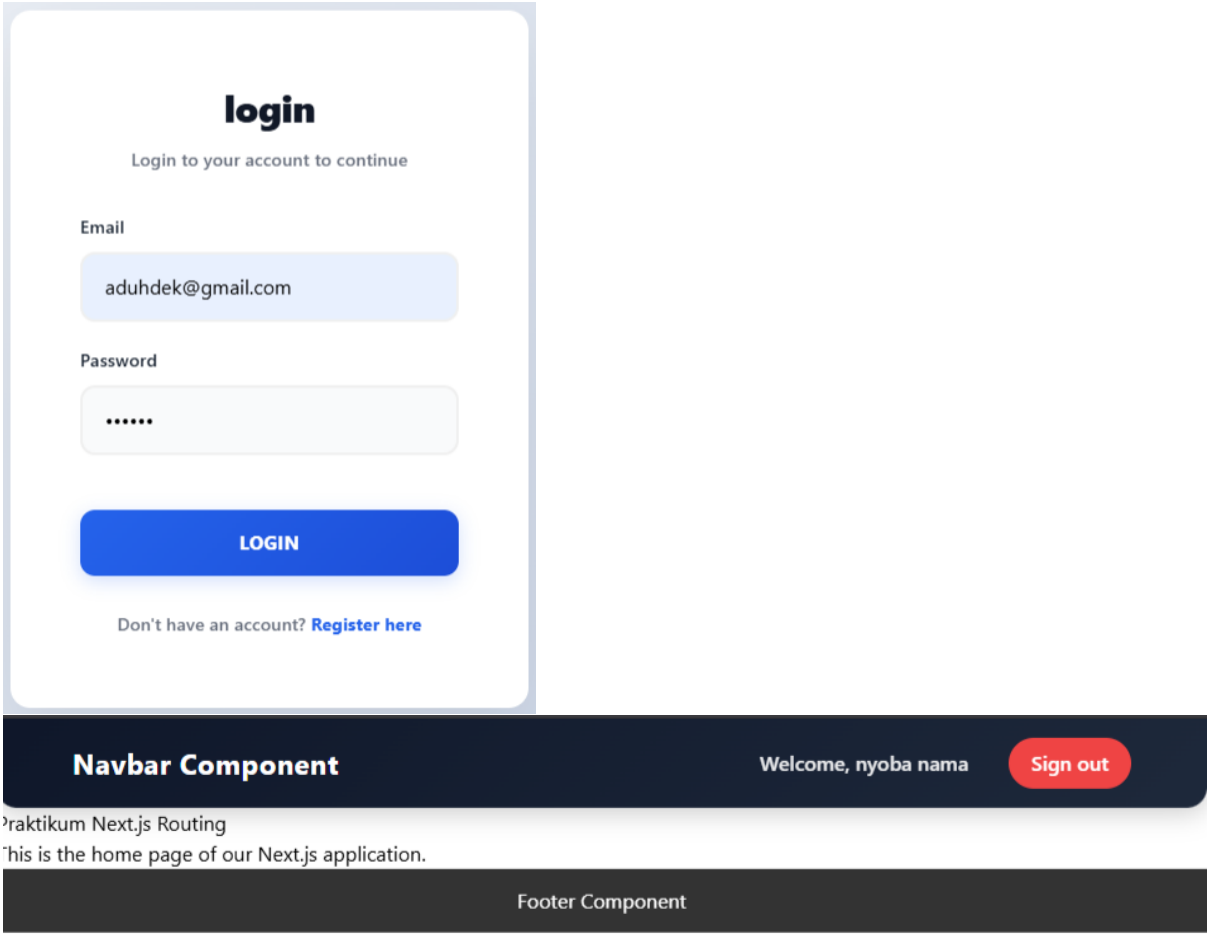
```
import NextAuth, { NextAuthOptions } from "next-auth";
import CredentialsProvider from "next-auth/providers/credentials";
import bcrypt from "bcryptjs";
import { signIn } from "@/utils/db/servicesfirebase";

export const authOptions: NextAuthOptions = {
  session: {
    strategy: "jwt",
  },
  secret: process.env.NEXTAUTH_SECRET,
  providers: [
    CredentialsProvider({
      name: "credentials",
      credentials: {
        // name: { label: "Name", type: "text" },
        email: { label: "Email", type: "email" },
        password: { label: "Password", type: "password" },
      },

      async authorize(credentials) {
        if (!credentials?.email || !credentials?.password) {
          return null;
        }
        /* eslint-disable @typescript-eslint/no-explicit-any */
        const user: any = await signIn(credentials.email);

        if (user) {
          const isPasswordValid = await bcrypt.compare(
            credentials.password,
            user.password,
          );

          if (isPasswordValid) {
            // Pastikan mengembalikan object user yang berisi
            return {
              id: user.id,
              email: user.email,
              fullName: user.fullName,
              role: user.role,
            };
          }
        }

        return null;
      },
    }),
  ],
```

BAGIAN 4 – Tambahkan Role ke Token

1. JWT Callback pada file [...nextauth].ts Modifikasi menjadi

```
callbacks: {
  async jwt({ token, account, profile, user }) {
    if (account?.provider === "credentials" && user) {
      token.email = user.email;
      token.fullName = user.fullName;
      token.role = user.role;
    }
    return token;
  },
  async session({ session, token }: any) {
    if (token.email) {
      session.user.email = token.email;
    }
    if (token.fullName) {
      session.user.fullName = token.fullName;
    }
    if (token.role) {
      session.user.role = token.role;
    }
    return session;
  },
},
```

2. Jalankan browser http://localhost:3000/auth/login



BAGIAN 5 – Callback URL Logic

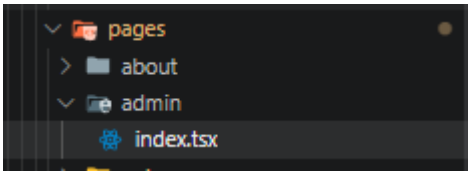
1. Modifikasi withAuth.ts pada folder src/middleware

P2 > Praktikum > next-routing > src > middleware > withAuth.ts > withAuth > <function>

```
import { getToken } from "next-auth/jwt";
import {
  NextFetchEvent,
  NextMiddleware,
  NextRequest,
  NextResponse,
} from "next/server";

export default function withAuth(
  middleware: NextMiddleware,
  requireAuth: string[] = []
) {
  return async (req: NextRequest, next: NextFetchEvent) => {
    const pathname = req.nextUrl.pathname;

    if (requireAuth.includes(pathname)) {
      const token = await getToken({
        req,
        secret: process.env.NEXTAUTH_SECRET,
      });

      if (!token) {
        const loginUrl = new URL("/", req.url);
        loginUrl.searchParams.set("callbackUrl", encodeURI(req.url));
        return NextResponse.redirect(loginUrl);
      }
    }

    return middleware(req, next);
  };
}
```

BAGIAN 6 – Membuat halaman Admin dan authoriz

1. Buat halaman admin



2. Pada index.tsx tambahkan code berikut

P2 > Praktikum > next-routing > src > pages > admin > index.tsx > default

```
const AdminPage = () => {
  return (
    <div>
      <h1>Admin Page</h1>
      <p>Welcome to the admin page! You can access every feature here.</p>
    </div>
  );
};

export default AdminPage;
```

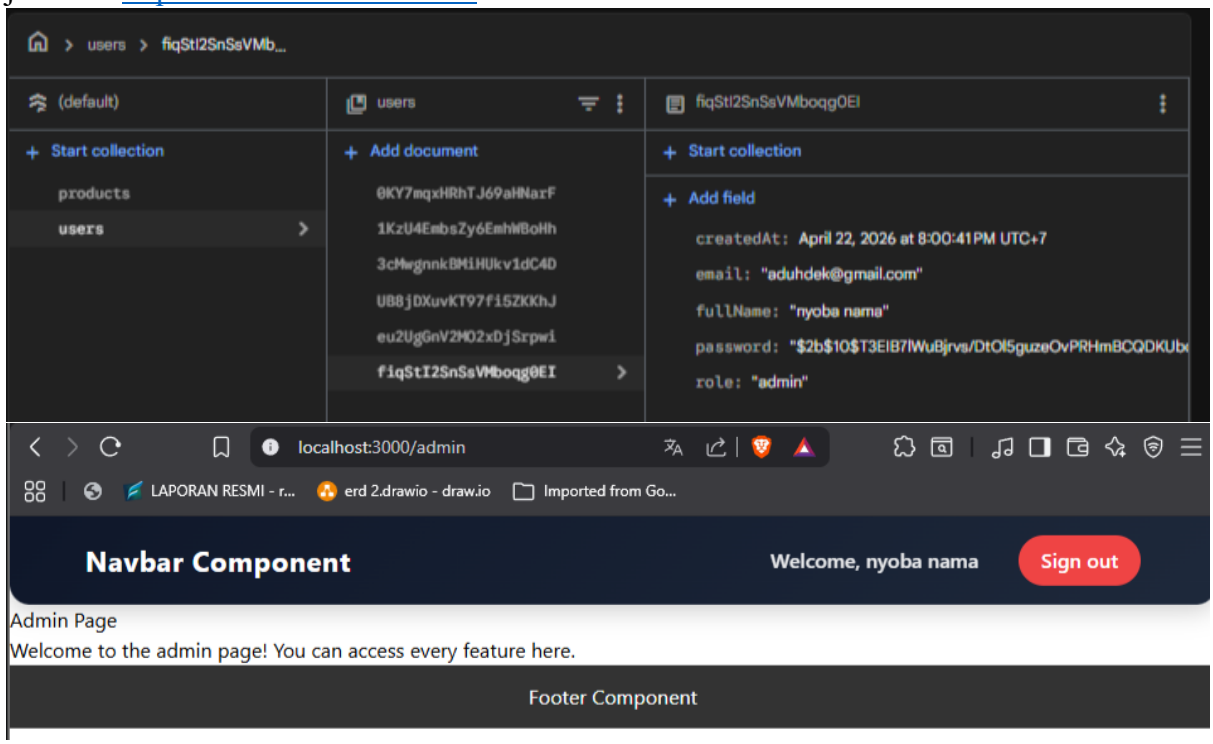
3. Modifikasi withAuth.ts

```
P2 > Praktikum > next-routing > src > middleware > withAuth.ts > withAuth > <function>
import { getToken } from "next-auth/jwt";
import {
  NextFetchEvent,
  NextMiddleware,
  NextRequest,
  NextResponse,
} from "next/server";

const hanyaAdmin = ["/admin"];

export default function withAuth(
  middleware: NextMiddleware,
  requireAuth: string[] = []
) {
  return async (req: NextRequest, next: NextFetchEvent) => {
    const pathname = req.nextUrl.pathname;

    if (requireAuth.includes(pathname)) {
      const token = await getToken({
        req,
        secret: process.env.NEXTAUTH_SECRET,
      });

      if (!token) {
        const loginUrl = new URL("/", req.url);
        loginUrl.searchParams.set("callbackUrl", encodeURI(req.url));
        return NextResponse.redirect(loginUrl);
      }
      if (token.role !== "admin" && hanyaAdmin.includes(pathname)) {
        return NextResponse.redirect(new URL("/", req.url));
      }
    }

    return middleware(req, next);
  };
}
```

4. Jalankan browser http://localhost:3000/product dan pada status sudah login. Rubah urlnya menjadi http://localhost:3000/admin maka user akan diarahkan ke localhost. Pada intinya role selain admin tidak bisa mengakses
   - Tambahkan kode untuk cek role admin di middleware.ts

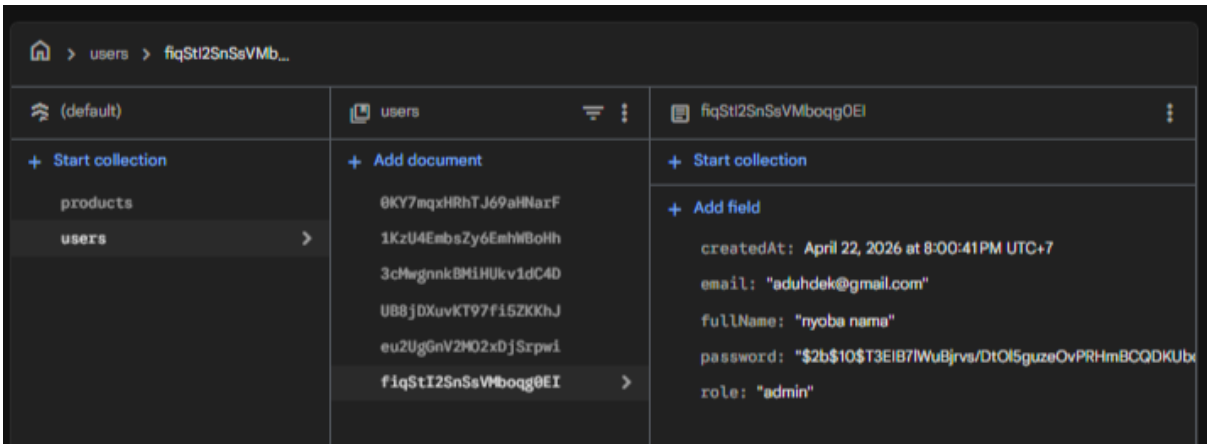
```
// Jika mengakses admin route, cek apakah user memiliki role admin
if (isAdminRoute) {
  if (!token) {
    // Jika belum login, redirect ke /auth/login
    return NextResponse.redirect(new URL("/auth/login", request.url));
  }
  if (token.role !== "admin") {
    // Jika sudah login tapi bukan admin, redirect ke home
    return NextResponse.redirect(new URL("/", request.url));
  }
}
```

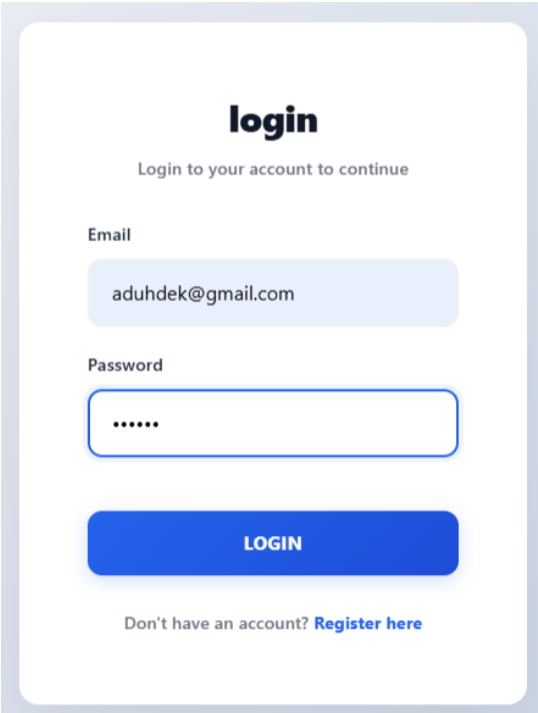
5. Untuk mencoba halaman admin rubah role pada firebas pada salah satu akun dan jalankan http://localhost:3000/admin



PENGUJIAN

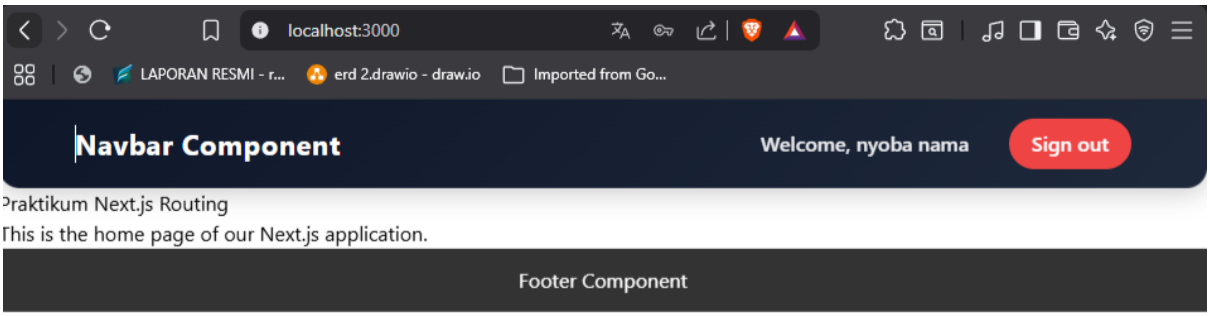
1. Uji 1 – Login Valid
   Input:
   - Email benar
   - Password benar





Hasil:

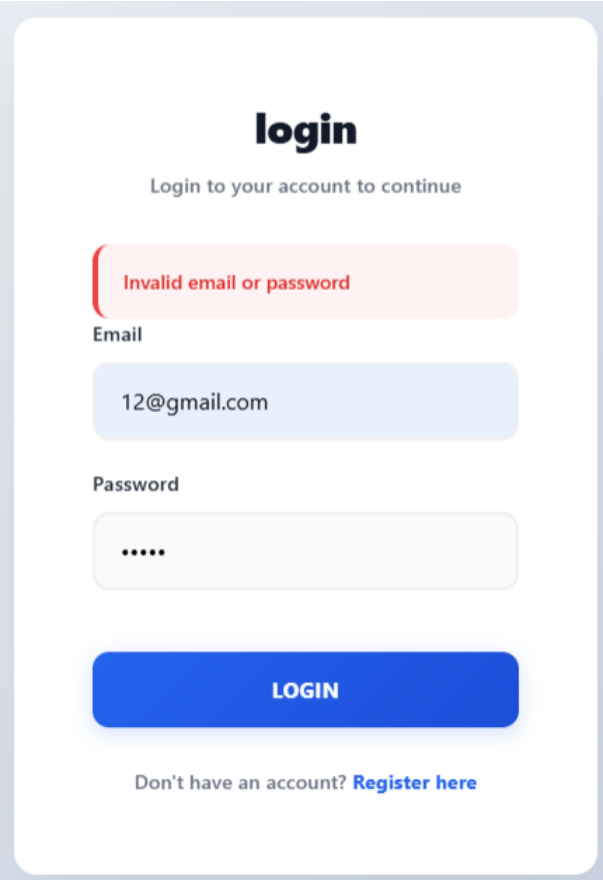
- Login berhasil
- Redirect sesuai callbackUrl



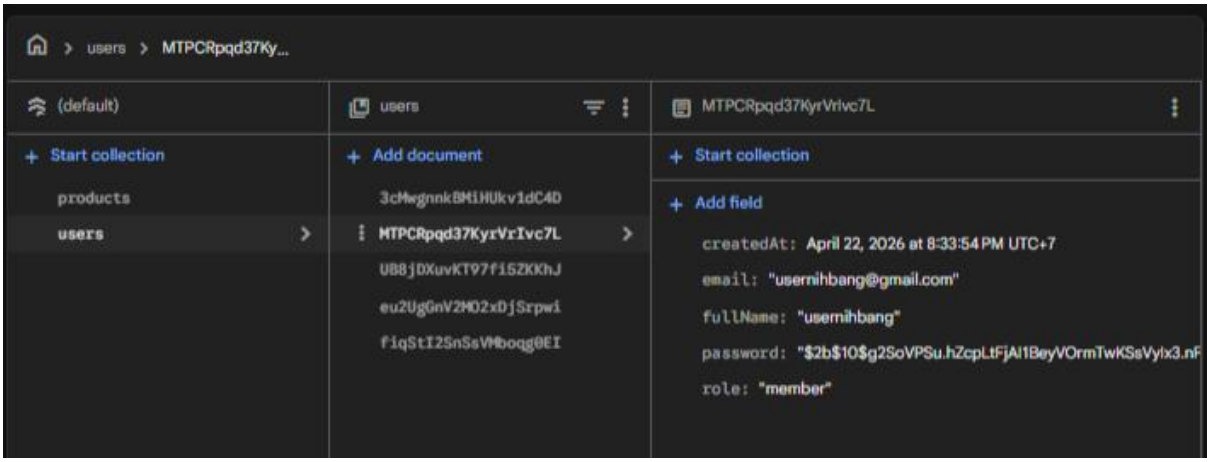

```
const handleSubmit = async (e: React.FormEvent<HTMLFormElement>) => {
  e.preventDefault();
  setIsLoading(true);

  try {
    const result = await signIn("credentials", {
      email: formData.email,
      password: formData.password,
      redirect: false,
    });

    if (result?.ok) {
      setFormData({ email: "", password: "" });
      setIsLoading(false);
      push("/");
    } else {
      setIsLoading(false);
      push(
        {
          pathname: "/auth/login",
          query: { error: "CredentialsSignin" },
        },
        undefined,
        { shallow: true },
      );
    }
  } catch (err) {
    setIsLoading(false);
    console.error("Login error:", err);
  }
};
```

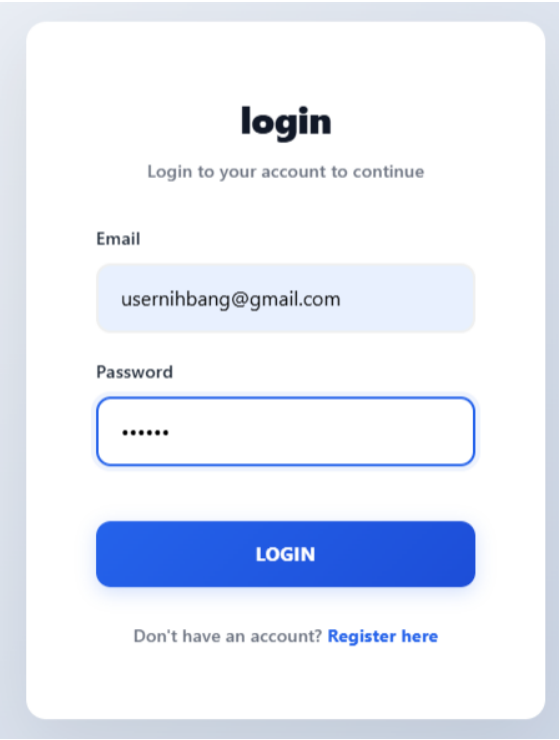
Sesuai dengan callbackUrl nya ke home “/”

2. Uji 2 – Password Salah
   Input:
   - Email benar
   - Password salah
   Hasil:
   - Error message tampil
   - Tidak login



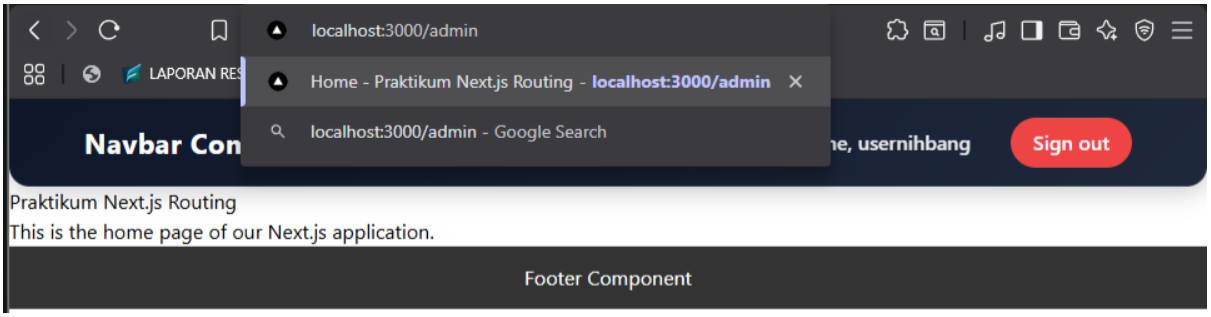
3. Uji 3 – Akses Admin sebagai User
   Login sebagai:
   - role: user



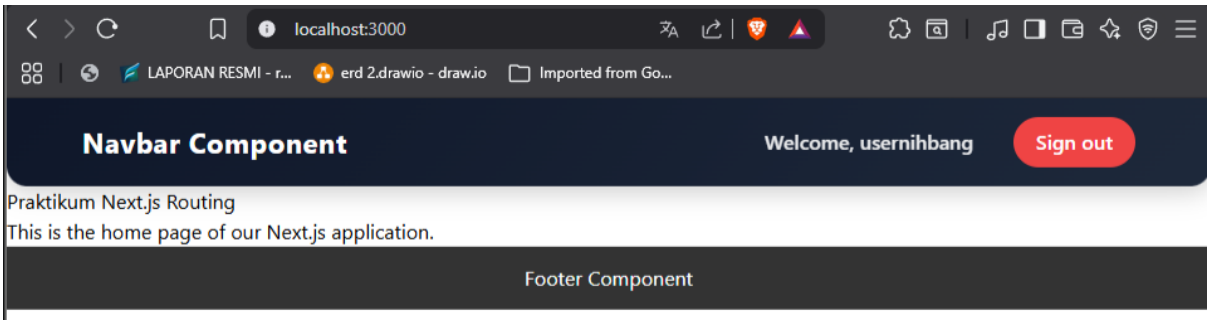


Akses: /admin
Hasil:

- Redirect ke home



Ketika di enter akan kembali ke home



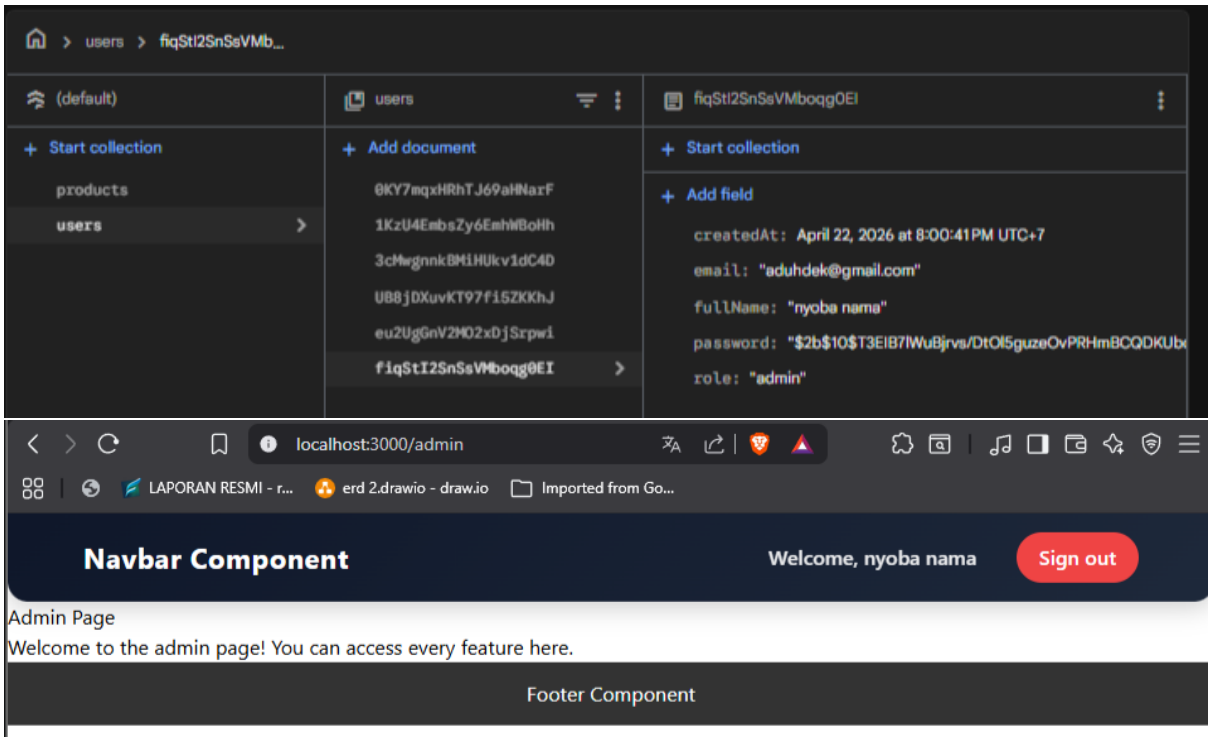
4. Uji 4 – Akses Admin sebagai Admin

   Login sebagai:

   - role: admin

   Akses: /admin
   Hasil:

   - Bisa masuk halaman admin



TUGAS PRAKTIKUM

1. Implementasikan login database.

```
export async function signIn(email: string) {
  try {
    console.log("Searching for user with email:", email);  ×8 ⁵ 'Searching for user with email:' | 'usernihbang@gmail.com' ⌛ ⁵ 'Searching f
    // Find user with this email
    const q = query(collection(db, "users"), where("email", "==", email));
    const querySnapshot = await getDocs(q);
    const users = querySnapshot.docs.map((doc) => ({
      id: doc.id,
      ...doc.data(),
    }));

    console.log("Users found:", users.length);  ×8 ⁵ 'Users found:' | 1 ⌛ ⁵ 'Users found:' | 1 ⌛ ⁵ 'Users found:' | 0 ⌛ ⁵ 'Users found:'

    // If user doesn't exist, return null
    if (users.length === 0) {
      console.log("No user found with email:", email);  ⁵ 'No user found with email:' | '12@gmail.com'
      return null;
    }

    /* eslint-disable @typescript-eslint/no-explicit-any */
    const user = users[0] as any;
    console.log("User data retrieved:", {  ×7 ⁵ 'User data retrieved:' | { id: 'MTPCRpqd37KyrVrIvc7L', email: 'usernihbang@gmail.com', fullN
      id: user.id,
      email: user.email,
      fullName: user.fullName,
      hasPassword: !!user.password,
    });

    return user;
  } catch (error) {
    console.error("Error during sign in:", error);
    return null;
  }
}
```

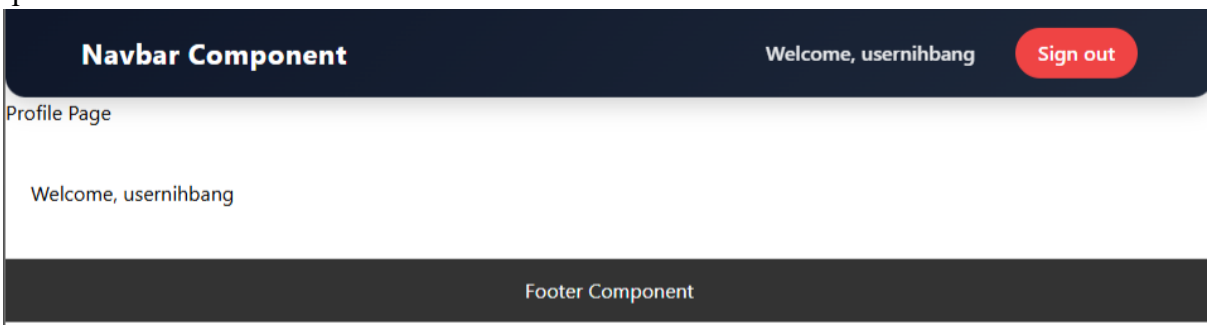
2. Tambahkan role pada user.

```
export async function signUp(
  userData: {
    email: string;
    password: string;
    fullName: string;
    role?: string;
  },
  callback: (response: {
    status: "success" | "error";
    message: string;
    userId?: string;
    error?: unknown;
  }) => void,
) {
```

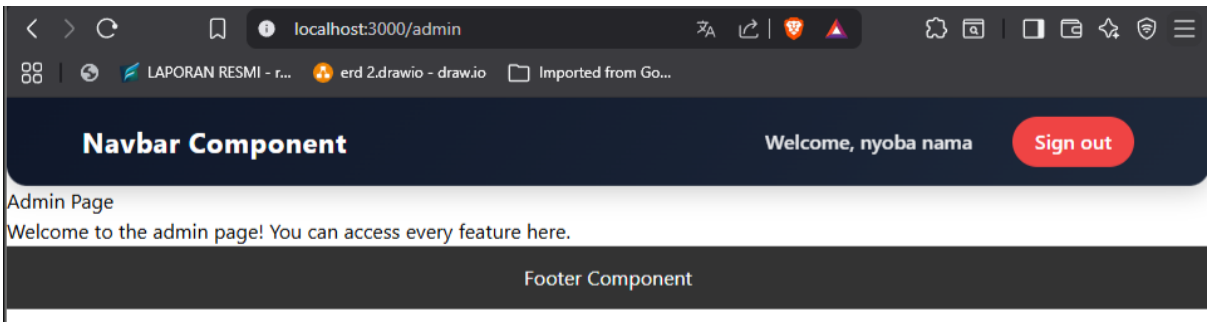
```
const newUser = await addDoc(collection(db, "users"), {
  email: userData.email,
  password: await bcrypt.hash(userData.password, 10),
  fullName: userData.fullName,
  role: userData.role || "member",
  createdAt: Timestamp.now(),
});
```

Pada waktu signUp akan menambahkan role secara otomatis menjadi “member”

3. Buat halaman:
   - /profile



   - /admin



4. Proteksi /admin hanya untuk admin.
   Semua ada di file middleware.ts

```
import { getToken } from "next-auth/jwt";
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

// Route yang memerlukan proteksi autentikasi
const protectedRoutes = ["/product", "/about"];
const profileRoutes = ["/profile"];
const adminRoutes = ["/admin"];
```

```
// Cek apakah route saat ini adalah admin
const isAdminRoute = adminRoutes.some((route) => pathname.startsWith(route));
```

```
// Jika mengakses admin route, cek apakah user memiliki role admin
if (isAdminRoute) {
  if (!token) {
    // Jika belum login, redirect ke /auth/login
    return NextResponse.redirect(new URL("/auth/login", request.url));
  }
  if (token.role !== "admin") {
    // Jika sudah login tapi bukan admin, redirect ke home
    return NextResponse.redirect(new URL("/", request.url));
  }
}
```

5. Implementasikan callback URL.

```
if (!token) {
  const loginUrl = new URL("/", req.url);
  loginUrl.searchParams.set("callbackUrl", encodeURI(req.url));
  return NextResponse.redirect(loginUrl);
}
```

PERTANYAAN ANALISIS

1. **Mengapa password harus diverifikasi dengan bcrypt.compare?**
   Password harus diverifikasi dengan bcrypt.compare karena password yang disimpan di database sudah dalam bentuk hash (bukan teks asli). Fungsi ini digunakan untuk mencocokkan password input user dengan hash tersebut secara aman, karena hashing bersifat one-way (tidak bisa dikembalikan ke bentuk asli), jadi tidak bisa dibandingkan langsung.

2. **Mengapa role disimpan di token?**
   Role disimpan di token (misalnya JWT) supaya setiap request tidak perlu query ulang ke database untuk mengetahui hak akses user. Dengan begitu, proses autentikasi dan otorisasi jadi lebih cepat dan efisien karena informasi role sudah ikut dibawa di token.

3. **Apa fungsi callbackUrl?**
   callbackUrl berfungsi untuk menentukan ke mana user akan diarahkan setelah login berhasil. Biasanya digunakan agar user kembali ke halaman yang sebelumnya ingin diakses sebelum login, sehingga pengalaman pengguna tetap seamless.

4. **Mengapa middleware penting untuk security?**
   Middleware penting untuk security karena bertindak sebagai "penjaga gerbang" sebelum request diproses lebih lanjut. Middleware bisa mengecek apakah user sudah login, apakah token valid, dan apakah user punya izin untuk mengakses route tertentu.

5. **Apa risiko jika role tidak dicek di middleware?**
   Jika role tidak dicek di middleware, maka semua user (termasuk yang tidak berhak) bisa mengakses fitur yang seharusnya dibatasi, seperti halaman admin. Ini bisa menyebabkan kebocoran data, manipulasi sistem, bahkan penyalahgunaan akses secara serius.